
Assets/Editor/CardEditor.cs ... Assets/Editor/DebugModeTools.cs

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEditor;

[CustomEditor(typeof(card))]
public class CardEditor : Editor
{
    public override void OnInspectorGUI()
    {
        base.OnInspectorGUI();
        var x = target as card;
        if (GUILayout.Button("show"))
        {
            x.show();
        }
        if (GUILayout.Button("hide"))
        {
            x.hide();
        }
    }
}

using UnityEditor;
using UnityEngine;

public class DebugModeTools : EditorWindow
{
    // 使用DebugManager中的静态变量来存储调试模式状态
    // 这样运行时脚本也可以访问

    [MenuItem("Tools/调试模式", false, 0)]
    private static void ToggleDebugMode()
    {
        bool newState = !DebugManager.IsDebugMode;
        DebugManager.IsDebugMode = newState;

        // 保存设置到EditorPrefs
        EditorPrefs.SetBool("DebugModeEnabled", newState);

        // 显示调试信息
        Debug.Log("调试模式已" + (DebugManager.IsDebugMode ? "开启" : "关闭"));

        // 重新绘制菜单，以便更新勾选状态
        //EditorApplication.RepaintMenus();
    }

    // 在菜单中显示勾选状态
    [MenuItem("Tools/调试模式", true)]
    private static bool ToggleDebugModeValidate()
    {

```

Assets/Editor/DebugModeTools.cs ... Assets/Editor/DraggableGridItemEditor.cs

```
        Menu.SetChecked("Tools/调试模式", DebugManager.IsDebugMode);
        return true;
    }
}

using UnityEngine;
using UnityEditor;

[CustomEditor(typeof(DraggableGridItem))]
public class DraggableGridItemEditor : Editor
{
    public override void OnInspectorGUI()
    {
        serializedObject.Update();

        // 获取当前编辑的 DraggableGridItem 对象
        DraggableGridItem item = (DraggableGridItem)target;

        SerializedProperty prop = serializedObject.GetIterator();

        // 绘制每个可见属性，但将 adjacency 字段以只读标签形式显示
        bool enterChildren = true;
        while (prop.NextVisible(enterChildren))
        {
            enterChildren = false;

            // 禁用对脚本字段的编辑
            if (prop.name == "m_Script")
            {
                EditorGUI.BeginDisabledGroup(true);
                EditorGUILayout.PropertyField(prop, true);
                EditorGUI.EndDisabledGroup();
                continue;
            }

            if (prop.name == "adjacentLeft" || prop.name == "adjacentRight" || prop.name == "adjacentTop" || prop.name == "adjacentBottom")
            {
                // 以标签形式显示布尔值，避免用户误改
                string niceName = ObjectNames.NicifyVariableName(prop.name);
                EditorGUILayout.LabelField(niceName, prop.boolValue ? "True" : "False");
            }
            else
            {
                EditorGUILayout.PropertyField(prop, true);
            }
        }

        // 显式显示 PositionIndex 属性
        EditorGUILayout.Space();
        EditorGUILayout.LabelField("Position Index", EditorStyles.boldLabel);
        EditorGUILayout.LabelField("Current Position Index", item.PositionIndex.ToString());
    }
}
```

Assets/Editor/DraggableGridItemEditor.cs ... Assets/Editor/frmbaseeditor.cs

```

        serializedObject.ApplyModifiedProperties();
    }
}

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEditor;

[CustomEditor(typeof(frmbase), true)]
public class frmbaseeditor : Editor
{
    // Start is called before the first frame update
    public override void OnInspectorGUI()
    {
        base.OnInspectorGUI();
        if (GUILayout.Button("显示"))
        {
            frm.show();
        }
        if (GUILayout.Button("隐藏"))
        {
            frm.hide();
        }

        // 检测是否存在gb节点
        Transform gbTransform = frm.transform.Find("Root");
        if (gbTransform == null)
        {
            // 如果没有gb节点, 显示初始化按钮
            if (GUILayout.Button("初始化 - 添加gb节点"))
            {
                InitializeGBNode();
            }
        }
        //if (GUILayout.Button("addString"))
        //{
        //    var sx = frm.GetComponentInChildren<TMPro.TextMeshProUGUI>(true);
        //    for (int i=0;i<sx.Length;i++)
        //    {
        //        var x = sx[i];
        //        var tx = x.GetComponent<stringinfo>();
        //        if (tx == null)
        //        {
        //            tx = x.gameObject.AddComponent<stringinfo>();
        //        }
        //        tx.getKey();
        //    }
        //}
        //if (GUILayout.Button("英文"))
        //{

```

Assets/Editor/frmbaseeditor.cs ... Assets/Editor/frmgameeditor.cs

```

        // LangData.SetLanguage(LangData.Language.English);
        // frm.loadString();
    //}
    //if (GUILayout.Button("中文"))
    //{
        // LangData.SetLanguage(LangData.Language.Chinese);
        // frm.loadString();
    //}
}

/// <summary>
/// 初始化gb节点
/// </summary>
private void InitializeGBNode()
{
    // 创建gb节点
    GameObject gbObj = new GameObject("gb");
    gbObj.transform.SetParent(frm.transform, false);

    // 添加RectTransform组件
    RectTransform rectTransform = gbObj.AddComponent<RectTransform>();

    // 设置全屏参数
    rectTransform.anchorMin = Vector2.zero;
    rectTransform.anchorMax = Vector2.one;
    rectTransform.offsetMin = Vector2.zero;
    rectTransform.offsetMax = Vector2.zero;
    rectTransform.localScale = Vector3.one;

    Debug.Log("已创建gb节点并设置为全屏");
}

frmbase frm
{
    get
    {
        return target as frmbase;
    }
}

}

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class frmgameeditor : MonoBehaviour
{
    // Start is called before the first frame update
    void Start()
    {

```

Assets/Editor/frmgameeditor.cs ... Assets/Editor/picmgrEditor.cs

```
}

// Update is called once per frame
void Update()
{

}

}

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEditor;

[CustomEditor(typeof(goldplay))]
public class goldplayeditor : Editor
{
    public override void OnInspectorGUI()
    {
        base.OnInspectorGUI();
        goldplay goldplay = (goldplay)target;
        if (GUILayout.Button("Goldplay"))
        {
            goldplay.Play();
        }
        if (GUILayout.Button("Goldplay2"))
        {
            goldplay.Trans();
        }
    }
}

using UnityEngine;
using UnityEditor;

[CustomEditor(typeof(picmgr))]
public class picmgrEditor : Editor
{
    // 难度参数相关字段
    private int level = 1;
    private int maxKeepCount = -1;
    private bool isHard = false;

    public override void OnInspectorGUI()
    {
        DrawDefaultInspector();

        picmgr picManager = (picmgr)target;

        EditorGUILayout.Space();

        // 显示难度参数设置
```

Assets/Editor/picmgrEditor.cs ... Assets/Editor/powereditor.cs

```

    EditorGUILayout.LabelField("难度设置", EditorStyles.boldLabel);
    level = EditorGUILayout.IntField("关卡编号", level);
    maxKeepCount = EditorGUILayout.IntField("最多原位数量 (-1为默认)", maxKeepCount);
    isHard = EditorGUILayout.Toggle("是否为困难模式", isHard);

    EditorGUILayout.Space();

    if (GUILayout.Button("生成网格图像"))
    {
        picManager.tCreateGridImages(level, maxKeepCount, isHard);
    }

    if (GUILayout.Button("打乱网格位置"))
    {
        picManager.ShuffleGridPositions(level, maxKeepCount, isHard);
    }

    if (GUILayout.Button("更新边框显示"))
    {
        picManager.UpdateBorderVisibility();
    }

    EditorGUILayout.HelpBox("点击上方按钮来生成网格图像。这将在当前对象下创建width*height个子对象，每个子对象包含一个RawImage组件， ...
}
}
using UnityEditor;
using UnityEngine;
using UnityEngine.UI;

[CustomEditor(typeof(power))]
public class powereditor : Editor
{
    private SerializedProperty valueProperty;

    private void OnEnable()
    {
        valueProperty = serializedObject.FindProperty("value");
    }

    public override void OnInspectorGUI()
    {
        serializedObject.Update();

        EditorGUI.BeginChangeCheck();
        EditorGUILayout.PropertyField(valueProperty, new GUIContent("Value", "0 shows all images, 1 hides all images. "));
        if (EditorGUI.EndChangeCheck())
        {
            valueProperty.floatValue = Mathf.Clamp01(valueProperty.floatValue);
            serializedObject.ApplyModifiedProperties();
            RefreshTargets();
        }
    }
}

```

Assets/Editor/powereditor.cs

```
    }
    else
    {
        serializedObject.ApplyModifiedProperties();
    }

    power powerTarget = (power)target;
    int imageCount = CountImages(powerTarget.transform);
    int showCount = GetShowCount(valueProperty.floatValue, imageCount);
    int activeCount = CountActiveImages(powerTarget.transform);

    EditorGUILayout.Space(8f);
    EditorGUILayout.LabelField("Image State", EditorStyles.boldLabel);
    EditorGUILayout.LabelField("Direct Child Images", imageCount.ToString());
    EditorGUILayout.LabelField("Active Images", activeCount + "/" + imageCount);

    Rect previewRect = GUILayoutUtility.GetRect(1f, 24f);
    DrawPreview(previewRect, imageCount, showCount);

    if (imageCount != 5)
    {
        EditorGUILayout.HelpBox("Direct child Image count is not 5. Please check the power prefab structure.", MessageType.Warning);
    }

    EditorGUILayout.Space(6f);
    DrawQuickButtons(imageCount);

    if (GUILayout.Button("Refresh View"))
    {
        RefreshTargets();
    }
}

private void DrawQuickButtons(int imageCount)
{
    EditorGUILayout.LabelField("Quick Set", EditorStyles.boldLabel);

    int count = Mathf.Max(imageCount, 5);
    for (int row = 0; row < 2; row++)
    {
        EditorGUILayout.BeginHorizontal();
        for (int column = 0; column < 3; column++)
        {
            int visibleCount = count - (row * 3 + column);
            if (visibleCount < 0)
            {
                GUILayout.FlexibleSpace();
                continue;
            }
        }
    }
}
```

Assets/Editor/powereditor.cs

```
        if (GUILayout.Button(visibleCount + "/" + count))
        {
            SetValue(count == 0 ? 1f : 1f - (float)visibleCount / count);
        }
    }
    EditorGUILayout.EndHorizontal();
}

private void SetValue(float value)
{
    Undo.RecordObjects(targets, "Set Power Value");
    foreach (Object item in targets)
    {
        power powerItem = (power)item;
        powerItem.Value = value;
        EditorUtility.SetDirty(powerItem);
    }

    serializedObject.Update();
}

private void RefreshTargets()
{
    foreach (Object item in targets)
    {
        power powerItem = (power)item;
        powerItem.Refresh();
        EditorUtility.SetDirty(powerItem);
    }
}

private static int CountImages(Transform root)
{
    int count = 0;
    for (int i = 0; i < root.childCount; i++)
    {
        if (root.GetChild(i).GetComponent<Image>() != null)
        {
            count++;
        }
    }

    return count;
}

private static int CountActiveImages(Transform root)
{
    int count = 0;
    for (int i = 0; i < root.childCount; i++)
```

Assets/Editor/powereditor.cs ... Assets/FollowTarget.cs

```
{
    Transform child = root.GetChild(i);
    if (child.GetComponent<Image>() != null && child.gameObject.activeSelf)
    {
        count++;
    }
}

return count;
}

private static int GetShowCount(float value, int imageCount)
{
    return Mathf.Clamp(Mathf.CeilToInt((1f - value) * imageCount), 0, imageCount);
}

private static void DrawPreview(Rect rect, int imageCount, int showCount)
{
    if (imageCount <= 0)
    {
        EditorGUI.LabelField(rect, "No direct child Image found.");
        return;
    }

    float gap = 3f;
    float width = (rect.width - gap * (imageCount - 1)) / imageCount;
    for (int i = 0; i < imageCount; i++)
    {
        Rect itemRect = new Rect(rect.x + (width + gap) * i, rect.y + 3f, width, rect.height - 6f);
        EditorGUI.DrawRect(itemRect, i < showCount ? new Color(0.25f, 0.75f, 0.35f) : new Color(0.28f, 0.28f, 0.28f));
    }
}

}

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using DG.Tweening;

public class FollowTarget : MonoBehaviour
{
    public Transform target;
    public float maxdis = 1000f;
    public float time = 1f;
    public float delaytime = 2f;
    public ParticleSystemForceField fd
    {
        get
        {
            return GetComponent<ParticleSystemForceField>();
        }
    }
}
```

Assets/FollowTarget.cs ... Assets/frm_msg.cs

```
public void Play()
{
    if (fd != null)
    {
        fd.endRange = 0; // 初始值设为0
        DOTween.To(() => fd.endRange, x => fd.endRange = x, maxdis, time).onComplete=()=>
        {
            DOVirtual.DelayedCall(delaytime, () =>
            {
                fd.endRange = 0;
            });
        };
    }
}

// Update is called once per frame
void Update()
{
    if(target)
        transform.position = target.position;
}
}

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using DG.Tweening;
using TMPro;

public class frm_msg : frmbase
{
    public GameObject msg;
    private void Awake()
    {
        Main.RegistEvent("event_msg", (x) =>
        {
            var xx = x as string;
            var t = GameObject.Instantiate(msg, gb);
            t.SetActive(true);
            t.GetComponent<TextMeshProUGUI>().text = xx;

            // 初始缩放为0
            t.transform.localScale = Vector3.zero;
            // 从0到1的缩放动画
            t.transform.DOScale(Vector3.one, 0.5f).OnComplete(() =>
            {
                // 停留0.5秒后执行后续动画
                DOVirtual.DelayedCall(0.5f, () =>
                {
                    // 向上移动并淡出的动画
                });
            });
        });
    }
}
```

Assets/frm_msg.cs ... Assets/frm_victory.cs

```

        var r = t.GetComponent<RectTransform>();
        var pos = r.anchoredPosition + new Vector2(0, 50);
        t.GetComponent<RectTransform>().DOAnchorPos(pos, 0.5f);
        t.GetComponent<TextMeshProUGUI>().DOFade(0, 0.5f).OnComplete(() =>
        {
            // 动画结束后销毁对象
            Destroy(t);
        });
    });
});

return 1;
});
}
}

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using DG.Tweening;
using TMPro;

public class frm_tips : frmbase
{
    public GameObject text;
    private void Awake()
    {
        Main.RegistEvent("event_tip", (x) =>
        {
            var xgb = GameObject.Instantiate(this.gameObject, gb);
            xgb.gameObject.SetActive(true);
            xgb.GetComponent<TextMeshProUGUI>().text = x as string;
            xgb.transform.DOScale(Vector3.one, 0.5f).OnComplete(() =>
            {
                xgb.transform.DOScale(Vector3.zero, 0.5f).OnComplete(() =>
                {
                    GameObject.Destroy(xgb);
                });
                xgb.GetComponent<RectTransform>().DOMoveY(20, 0.5f).SetEase(Ease.OutBack);
            });
            return 1;
        });
    }
}

using cfg;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;

public class frm_victory : frmbase
{

```

Assets/frm_victory.cs ... Assets/getpos.cs

```
public RawImage img;
public Button close;
private void Awake()
{
    Main.RegistEvent("show_next", (x)=>
    {
        Debug.Log("show_next");
        show();

        DrLevel leevel = x as DrLevel;
        img.texture = Resources.Load(leevel.LevelFigure) as Texture2D;

        return null;
    });
    close.onClick.AddListener(()=>
    {
        Main.SendEvent("level_next");
        hide();
    });
}
// Start is called before the first frame update
void Start()
{

}

// Update is called once per frame
void Update()
{

}
}
using Coffee.UIExtensions;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class getpos : MonoBehaviour
{
    private ParticleSystem.Particle[] particles;
    public Transform tg;
    // Start is called before the first frame update
    void Start()
    {
        particles = new ParticleSystem.Particle[p.main.maxParticles];
    }

    ParticleSystem p
    {
        get
```

Assets/getpos.cs ... Assets/goldplay.cs

```

        {
            return GetComponent<ParticleSystem>();
        }
    }

    // Update is called once per frame
    void Update()
    {

        int count = p.GetParticles(particles);
        if(count > 0)
        {
            tg.transform.position = particles[0].position;
        }
    }
}

using Coffee.UIExtensions;
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using DG.Tweening;
using UnityEngine.UI;

public class goldplay : MonoBehaviour
{
    public float delay = 1f;
    public void Play()
    {
        tagmt.endRange = 0;
        GetComponent<UIParticle>().Stop();
        GetComponent<UIParticle>().Play();
        DOVirtual.DelayedCall(delay, () =>
        {

            RectTransform canvasRect = canvas.transform as RectTransform;
            RectTransformUtility.ScreenPointToWorldPointInRectangle(
                canvasRect,
                targetIcon.position,
                null, // Overlay模式无需相机
                out targetWorldPos
            );
            tagmt.transform.position = targetWorldPos;
            var local = ptk.transform.InverseTransformPoint(targetWorldPos);
            tagmt.transform.SetParent(ptk.transform);
            tagmt.transform.localPosition /= (10 / 0.66666666667f);
            tagmt.endRange = 180;
        });
    }

    public void Trans()

```

Assets/goldplay.cs ... Assets/ParticleScreenPosConverter.cs

```
{

}

private ParticleSystem ptk
{
    get
    {
        return GetComponentInChildren<ParticleSystem>();
    }
}

private int maxParticles; // 缓存最大粒子数

private void Awake()
{
    if (ptk != null)
    {
        maxParticles = ptk.main.maxParticles;
        particles = new ParticleSystem.Particle[maxParticles];
    }
}

// 添加死亡距离阈值
public float deathDistance = 0.1f;

// 添加力的衰减因子
public float forceAttenuation = 1.0f;

// 吸引力强度
public float attractForce = 1;

private Vector3 targetWorldPos; // 目标图标的世界坐标
public Transform targetIcon; // 目标UI图标 (Image/Button)
public Canvas canvas; // 主Canvas (Overlay模式)
public Image follow;

public ParticleSystemForceField tagmt;

public float speed = 1f;
private ParticleSystem.Particle[] particles;
public float flySpeed = 500;
public float drag = 5f;
}

using Coffee.UIExtensions;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

/// <summary>
/// UIParticle粒子屏幕坐标转换工具
```

Assets/ParticleScreenPosConverter.cs

```
/// 解决UIParticle中粒子位置不是真实世界坐标的问题
/// </summary>
public class ParticleScreenPosConverter : MonoBehaviour
{
    [Header("组件引用")]
    public ParticleSystem uiParticle;    // UIParticle组件
    public Canvas targetCanvas;         // 目标Canvas
    public Camera uiCamera;             // UI相机（仅在ScreenSpace-Camera模式下需要）

    [Header("调试")]
    public Transform debugTarget;        // 调试用的目标对象，用于显示第一个粒子的位置

    private ParticleSystem.Particle[] particlesBuffer;

    void Start()
    {
        // 初始化粒子缓冲区
        if (uiParticle != null)
        {
            particlesBuffer = new ParticleSystem.Particle[uiParticle.main.maxParticles];
        }
    }

    void Update()
    {
        if (uiParticle == null || targetCanvas == null) return;

        // 获取活跃粒子数量
        int activeCount = uiParticle.GetParticles(particlesBuffer);
        if (activeCount <= 0) return;

        // 获取第一个粒子的屏幕坐标
        Vector2 screenPos = GetParticleScreenPosition(0);

        // 可选：在场景中显示第一个粒子的位置
        if (debugTarget != null)
        {
            debugTarget.position = ScreenToWorld(screenPos);
        }

        // 打印第一个粒子的屏幕坐标
        Debug.Log($"第一个粒子的屏幕坐标: {screenPos}");
    }

    /// <summary>
    /// 获取指定索引粒子的屏幕坐标
    /// </summary>
    /// <param name="particleIndex">粒子索引</param>
    /// <returns>屏幕坐标</returns>
    public Vector2 GetParticleScreenPosition(int particleIndex)
```

Assets/ParticleScreenPosConverter.cs

```
{
    // 1. 校验参数
    if (uiParticle == null || targetCanvas == null) return Vector2.zero;

    // 2. 获取粒子数据
    if (particlesBuffer == null || particlesBuffer.Length < uiParticle.main.maxParticles)
    {
        particlesBuffer = new ParticleSystem.Particle[uiParticle.main.maxParticles];
    }

    int activeCount = uiParticle.GetParticles(particlesBuffer);
    if (particleIndex < 0 || particleIndex >= activeCount) return Vector2.zero;

    // 3. 获取粒子的世界坐标（注意：这是UIParticle的特殊坐标）
    Vector3 particleWorldPos = particlesBuffer[particleIndex].position;

    // 4. 根据Canvas渲染模式转换为屏幕坐标
    return WorldToScreenPoint(particleWorldPos);
}

/// <summary>
/// 将世界坐标转换为屏幕坐标（适配不同Canvas模式）
/// </summary>
/// <param name="worldPos">世界坐标</param>
/// <returns>屏幕坐标</returns>
private Vector2 WorldToScreenPoint(Vector3 worldPos)
{
    Vector2 screenPos = Vector2.zero;

    switch (targetCanvas.renderMode)
    {
        // 最常用：ScreenSpace-Overlay模式（2D UI）
        case RenderMode.ScreenSpaceOverlay:
            screenPos = RectTransformUtility.WorldToScreenPoint(null, worldPos);
            // 注意：Unity的屏幕坐标Y轴原点在左下角，而Canvas的Y轴原点在左上角
            // 但在大多数情况下不需要手动转换，RectTransformUtility已经处理了
            break;

        // ScreenSpace-Camera模式（带相机的UI）
        case RenderMode.ScreenSpaceCamera:
            if (uiCamera == null) uiCamera = targetCanvas.worldCamera;
            screenPos = RectTransformUtility.WorldToScreenPoint(uiCamera, worldPos);
            break;

        // WorldSpace模式（3D UI）
        case RenderMode.WorldSpace:
            // WorldSpace模式下的UI通常不需要转换为屏幕坐标
            screenPos = RectTransformUtility.WorldToScreenPoint(Camera.main, worldPos);
            break;
    }
}
```

Assets/ParticleScreenPosConverter.cs

```
        return screenPos;
    }

    /// <summary>
    /// 将屏幕坐标转换为世界坐标（用于调试显示）
    /// </summary>
    /// <param name="screenPos">屏幕坐标</param>
    /// <returns>世界坐标</returns>
    private Vector3 ScreenToWorld(Vector2 screenPos)
    {
        // 创建一个距离相机10单位的屏幕点
        Vector3 screenPoint = new Vector3(screenPos.x, screenPos.y, 10f);

        switch (targetCanvas.renderMode)
        {
            case RenderMode.ScreenSpaceOverlay:
                return Camera.main.ScreenToWorldPoint(screenPoint);

            case RenderMode.ScreenSpaceCamera:
                if (uiCamera == null) uiCamera = targetCanvas.worldCamera;
                return uiCamera.ScreenToWorldPoint(screenPoint);

            case RenderMode.WorldSpace:
                return Camera.main.ScreenToWorldPoint(screenPoint);

            default:
                return Camera.main.ScreenToWorldPoint(screenPoint);
        }
    }

    /// <summary>
    /// 获取粒子在Canvas中的局部坐标
    /// </summary>
    /// <param name="particleIndex">粒子索引</param>
    /// <returns>Canvas局部坐标</returns>
    public Vector2 GetParticleCanvasLocalPosition(int particleIndex)
    {
        // 1. 校验参数
        if (uiParticle == null || targetCanvas == null) return Vector2.zero;

        // 2. 获取粒子数据
        if (particlesBuffer == null || particlesBuffer.Length < uiParticle.main.maxParticles)
        {
            particlesBuffer = new ParticleSystem.Particle[uiParticle.main.maxParticles];
        }

        int activeCount = uiParticle.GetParticles(particlesBuffer);
        if (particleIndex < 0 || particleIndex >= activeCount) return Vector2.zero;
    }
}
```

Assets/ParticleScreenPosConverter.cs

```
// 3. 获取粒子的世界坐标
Vector3 particleWorldPos = particlesBuffer[particleIndex].position;

// 4. 转换为Canvas局部坐标
return WorldPosToCanvasLocal(particleWorldPos);
}

/// <summary>
/// 世界坐标转Canvas局部坐标
/// </summary>
/// <param name="worldPos">世界坐标</param>
/// <returns>Canvas局部坐标</returns>
private Vector2 WorldPosToCanvasLocal(Vector3 worldPos)
{
    Vector2 canvasLocalPos = Vector2.zero;
    RectTransform canvasRect = targetCanvas.transform as RectTransform;

    switch (targetCanvas.renderMode)
    {
        case RenderMode.ScreenSpaceOverlay:
            // 世界坐标→屏幕坐标→Canvas局部坐标
            Vector2 screenPos = RectTransformUtility.WorldToScreenPoint(null, worldPos);
            RectTransformUtility.ScreenPointToLocalPointInRectangle(
                canvasRect,
                screenPos,
                null, // Overlay模式无需相机
                out canvasLocalPos
            );
            break;

        case RenderMode.ScreenSpaceCamera:
            if (uiCamera == null) uiCamera = targetCanvas.worldCamera;
            Vector2 screenPosCam = RectTransformUtility.WorldToScreenPoint(uiCamera, worldPos);
            RectTransformUtility.ScreenPointToLocalPointInRectangle(
                canvasRect,
                screenPosCam,
                uiCamera,
                out canvasLocalPos
            );
            break;

        case RenderMode.WorldSpace:
            // 直接将世界坐标转换为Canvas局部坐标
            Vector3 localPos = targetCanvas.transform.InverseTransformPoint(worldPos);
            canvasLocalPos = new Vector2(localPos.x, localPos.y);
            break;
    }

    return canvasLocalPos;
}
```

Assets/ParticleScreenPosConverter.cs ... Assets/power.cs

```
}
using UnityEngine;
using UnityEngine.UI;

[ExecuteAlways]
public class power : MonoBehaviour
{
    [Range(0f, 1f)]
    [SerializeField] private float value;

    public float Value
    {
        get
        {
            return value;
        }
        set
        {
            this.value = Mathf.Clamp01(value);
            Refresh();
        }
    }

    private void Awake()
    {
        Refresh();
    }

    private void OnEnable()
    {
        Refresh();
    }

#if UNITY_EDITOR
    private void OnValidate()
    {
        value = Mathf.Clamp01(value);
        Refresh();
    }
#endif

    public void Refresh()
    {
        int imageCount = 0;
        for (int i = 0; i < transform.childCount; i++)
        {
            if (transform.GetChild(i).GetComponent<Image>() != null)
            {
                imageCount++;
            }
        }
    }
}
```

Assets/power.cs ... Assets/Scripts/BorderSpriteLoader.cs

```
    }

    int showCount = Mathf.Clamp(Mathf.CeilToInt((1f - value) * imageCount), 0, imageCount);
    int imageIndex = 0;
    for (int i = 0; i < transform.childCount; i++)
    {
        Transform child = transform.GetChild(i);
        if (child.GetComponent<Image>() == null)
        {
            continue;
        }

        child.gameObject.SetActive(imageIndex < showCount);
        imageIndex++;
    }
}

using System;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;

// 在运行时从 Resources 加载用于边框的贴图（优先 Sprite，然后尝试 Texture2D）
[RequireComponent(typeof(Image))]
public class BorderSpriteLoader : MonoBehaviour
{
    [Tooltip("资源名（在 Resources 下），例如 'lt','t','rt','r','rb','b','lb','l' 等")]
    public string resourceName;

    [Tooltip("当找不到资源时是否保留颜色占位（Image.color）")]
    public bool keepColorIfMissing = true;

    void Awake()
    {
    }

    internal void LoadSprite()
    {
        if (string.IsNullOrEmpty(resourceName)) return;

        Image img = GetComponent<Image>();
        if (img == null) return;
        // img.material = new Material(Shader.Find("Hidden/imagenec"));
        // 静态缓存，避免重复加载
        if (spriteCache == null)
            spriteCache = new Dictionary<string, Sprite>();

        Sprite s = null;
        if (spriteCache.TryGetValue(resourceName, out s)) {
```

Assets/Scripts/BorderSpriteLoader.cs

```

        img.sprite = s;
        img.color = Color.white;
    } else {
        string resourceNameNew = "Card/" + resourceName;
        s = Resources.Load<Sprite>(resourceNameNew);
        if (s != null) {
            img.sprite = s;
            img.color = Color.white;
            spriteCache[resourceNameNew] = s;
        } else {
            img.sprite = null;
            img.color = keepColorIfMissing ? Color.gray : Color.clear;
        }
    }
}

// 加载对应的mask贴图（添加m_前缀）
// string maskResourceName = "m_" + resourceName;
// if (img.material != null) {
//     Texture2D maskTexture = Resources.Load<Texture2D>(maskResourceName);
//     if (maskTexture != null) {
//         img.material.SetTexture("_Mask", maskTexture);
//     }
// }
// }

}

// 静态缓存，避免重复加载
private static Dictionary<string, Sprite> spriteCache;

// 直接设置资源名并加载
public void SetResourceAndLoad(string key)
{
    resourceName = key;
    LoadSprite();
}

// 解析父对象名称里保存的格子坐标，并尝试读取父级的 picmgr 网格尺寸
private void GetCellAndGrid(out int cellX, out int cellY, out int gridW, out int gridH)
{
    cellX = -1; cellY = -1; gridW = -1; gridH = -1;
    if (transform.parent != null)
    {
        string pname = transform.parent.gameObject.name.Replace("GridCell_", "");
        string[] parts = pname.Split('_');
        if (parts.Length == 2)
        {
            int.TryParse(parts[0], out cellX);
            int.TryParse(parts[1], out cellY);
        }
    }
}

picmgr mgr = GetComponentInParent<picmgr>();

```

Assets/Scripts/BorderSpriteLoader.cs

```
        if (mgr != null)
        {
            gridW = mgr.width;
            gridH = mgr.height;
        }
    }

    internal void RefreshBasedOnFlags(string prop, bool isTopCorrect, bool isLeftCorrect, bool isRightCorrect, bool isBottomCorrect, bool ...
    {
        // 只处理四个角的逻辑, 假定 resourceName 已经是角的类型 (lt, rt, rb, lb)
        string key = resourceName;
        switch (prop)
        {
            case "lt": // 左上角
                if (isLeftCorrect && isTopCorrect && !isConnerCorrect)
                    key = "lt_revert";
                else if (!isLeftCorrect && !isTopCorrect)
                    key = "lt";
                else if (isLeftCorrect && !isTopCorrect)
                    key = "t";
                else if (!isLeftCorrect && isTopCorrect)
                    key = "l";
                else
                {
                }

                break;
            case "rt": // 右上角
                if (isRightCorrect && isTopCorrect && !isConnerCorrect)
                    key = "rt_revert";
                else if (!isRightCorrect && !isTopCorrect)
                    key = "rt";
                else if (isRightCorrect && !isTopCorrect)
                    key = "t";
                else if (!isRightCorrect && isTopCorrect)
                    key = "r";
                else
                {
                }

                break;
            case "rb": // 右下角
                // 修改: 添加对右下方对角线邻居的判断
                if (isRightCorrect && isBottomCorrect && !isConnerCorrect)
                    key = "rb_revert";
                else if (!isRightCorrect && !isBottomCorrect)
                    key = "rb";
                else if (isRightCorrect && !isBottomCorrect)
                    key = "b";
                else if (!isRightCorrect && isBottomCorrect)
```

Assets/Scripts/BorderSpriteLoader.cs ... Assets/Scripts/card.cs

```

        key = "r";
    else
    {

    }
    break;
case "lb": // 左下角
    if (isLeftCorrect && isBottomCorrect && !isConnerCorrect)
        key = "lb_revert";
    else if (!isLeftCorrect && !isBottomCorrect)
        key = "lb";
    else if (isLeftCorrect && !isBottomCorrect)
        key = "b";
    else if (!isLeftCorrect && isBottomCorrect)
        key = "l";
    else
    {
    }
    break;
default:
    Debug.LogError("Invalid property: " + prop);
    break;
}
SetResourceAndLoad(key);
}
}

using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using TMPPro;
using DG.Tweening;
public class card : MonoBehaviour
{
    public Texture2D back;
    internal int levelid;
    internal float uvX;
    internal float uvY;
    internal float uvWidth;
    internal float uvHeight;
    internal Texture texture;
    public Sprite blocktex, frame;
    public Image src;
    public TextMeshProUGUI level;
    RawImage rawImage
    {
        get
        {
            return GetComponent<RawImage>();
        }
    }
}

```

Assets/Scripts/card.cs

```

    }
}

public bool isTurning = false;

internal void Load()
{
    if(levelid< PlayerData.gd.levelid )
    {
        if (PlayerData.gd.isOpened(levelid))
        {
            rawImage.uvRect = new Rect(uvX, (1 - uvY - uvHeight), uvWidth, uvHeight);
            rawImage.texture = texture;
            //level.gameObject.SetActive(false);
            show();
        }
        else
        {
            DOVirtual.DelayedCall(1.Of, () =>
            {
                rawImage.uvRect = new Rect(0, 0, 1, 1);
                rawImage.texture = back;
                //level.gameObject.SetActive(false);
                isTurning = true;
                transform.DOScaleX(0, .5f).onComplete = () =>
                {
                    transform.localScale = new Vector3(0, 1, 1);

                    rawImage.uvRect = new Rect(uvX, (1 - uvY - uvHeight), uvWidth, uvHeight);
                    rawImage.texture = texture;
                    show();
                    transform.DOScaleX(1, 0.5f).onComplete = () =>
                    {
                        transform.localScale = new Vector3(1, 1, 1);
                        isTurning = false;
                    };
                };
                var x = transform.localPosition.x;
                float wid = transform.GetComponent<RectTransform>().rect.width;
                PlayerData.gd.Open(levelid);
            });
            hide();
        }
    }
    else
    {
        rawImage.uvRect = new Rect(0, 0, 1, 1);
        rawImage.texture = back;
        //level.gameObject.SetActive(true);
        hide();
    }
}

```

Assets/Scripts/card.cs ... Assets/Scripts/Config/beans/IdValue.cs

```
    }

    public void show()
    {
        this.src.sprite = frame;
    }

    public void hide()
    {
        this.src.sprite = blocktex;
    }
}

//-----
// <auto-generated>
//     This code was generated by a tool.
//     Changes to this file may cause incorrect behavior and will be lost if
//     the code is regenerated.
// </auto-generated>
//-----

using Luban;
using SimpleJSON;

namespace cfg.beans
{
    public sealed partial class IdValue : Luban.BeanBase
    {
        public IdValue(JSONNode _buf)
        {
            { if(!_buf["id"].IsNumber) { throw new SerializationException(); } Id = _buf["id"]; }
            { if(!_buf["value"].IsNumber) { throw new SerializationException(); } Value = _buf["value"]; }
        }

        public static IdValue DeserializeIdValue(JSONNode _buf)
        {
            return new beans.IdValue(_buf);
        }

        /// <summary>
        /// ID
        /// </summary>
        public readonly int Id;
        /// <summary>
        /// 值
        /// </summary>
        public readonly int Value;

        public const int __ID__ = -1008851765;
    }
}
```

Assets/Scripts/Config/beans/IdValue.cs ... Assets/Scripts/Config/beans/Item.cs

```
public override int GetTypeId() => __ID__;

public void ResolveRef(Tables tables)
{
}

public override string ToString()
{
    return "{ "
        + "id:" + Id + ", "
        + "value:" + Value + ", "
        + "}";
}
}
}

//-----
// <auto-generated>
//     This code was generated by a tool.
//     Changes to this file may cause incorrect behavior and will be lost if
//     the code is regenerated.
// </auto-generated>
//-----

using Luban;
using SimpleJSON;

namespace cfg.beans
{
public sealed partial class Item : Luban.BeanBase
{
    public Item(JSONNode _buf)
    {
        { if(!_buf["itemId"].IsNumber) { throw new SerializationException(); } itemId = _buf["itemId"]; }
        { if(!_buf["itemNum"].IsNumber) { throw new SerializationException(); } itemNum = _buf["itemNum"]; }
    }

    public static Item DeserializeItem(JSONNode _buf)
    {
        {
            return new beans.Item(_buf);
        }
    }

    /// <summary>
    /// 道具Id
    /// </summary>
    public readonly int itemId;
    /// <summary>
    /// 道具数量

```

Assets/Scripts/Config/beans/Item.cs ... Assets/Scripts/Config/DrChapter.cs

```

    /// </summary>
    public readonly int ItemNum;

    public const int __ID__ = -515569794;
    public override int GetTypeId() => __ID__;

    public void ResolveRef(Tables tables)
    {
    }

    public override string ToString()
    {
        return "{ "
            + "itemId:" + ItemId + ", "
            + "itemNum:" + ItemNum + ", "
            + "}";
    }
}
}

//-----
// <auto-generated>
//     This code was generated by a tool.
//     Changes to this file may cause incorrect behavior and will be lost if
//     the code is regenerated.
// </auto-generated>
//-----

using Luban;
using SimpleJSON;

namespace cfg
{
    public sealed partial class DrChapter : Luban.BeanBase
    {
        public DrChapter(JSONNode _buf)
        {
            { if(!_buf["id"].IsNumber) { throw new SerializationException(); } Id = _buf["id"]; }
            { if(!_buf["chapter_title"].IsString) { throw new SerializationException(); } ChapterTitle = _buf["chapter_title"]; }
            { if(!_buf["chapter_figure"].IsString) { throw new SerializationException(); } ChapterFigure = _buf["chapter_figure"]; }
            { if(!_buf["chapter_figure_x"].IsNumber) { throw new SerializationException(); } ChapterFigureX = _buf["chapter_figure_x"]; }
            { if(!_buf["chapter_figure_y"].IsNumber) { throw new SerializationException(); } ChapterFigureY = _buf["chapter_figure_y"]; }
            { if(!_buf["next_chapter"].IsNumber) { throw new SerializationException(); } NextChapter = _buf["next_chapter"]; }
            { var __json0 = _buf["level_id"]; if(!__json0.IsArray) { throw new SerializationException(); } LevelId = new System.Collections. ...
        }

        public static DrChapter DeserializeDrChapter(JSONNode _buf)
        {

```

Assets/Scripts/Config/DrChapter.cs

```

        return new DrChapter(_buf);
    }

    /// <summary>
    /// 章节id
    /// </summary>
    public readonly int Id;
    /// <summary>
    /// 章节名称<br/> (多语言key)
    /// </summary>
    public readonly string ChapterTitle;
    /// <summary>
    /// 章节图片<br/> (配置资源名称)
    /// </summary>
    public readonly string ChapterFigure;
    /// <summary>
    /// x轴切分数量
    /// </summary>
    public readonly int ChapterFigureX;
    /// <summary>
    /// y轴切分数量
    /// </summary>
    public readonly int ChapterFigureY;
    /// <summary>
    /// 下一章id
    /// </summary>
    public readonly int NextChapter;
    /// <summary>
    /// 章节中关卡id
    /// </summary>
    public readonly System.Collections.Generic.List<int> LevelId;

    public const int __ID__ = 867732415;
    public override int GetTypeId() => __ID__;

    public void ResolveRef(Tables tables)
    {
    }

    public override string ToString()
    {
        return "{ "
            + "id:" + Id + ", "
            + "chapterTitle:" + ChapterTitle + ", "
            + "chapterFigure:" + ChapterFigure + ", "
            + "chapterFigureX:" + ChapterFigureX + ", "
            + "chapterFigureY:" + ChapterFigureY + ", "
            + "nextChapter:" + NextChapter + ", "
            + "levelId:" + Luban.StringUtil.CollectionToString(LevelId) + ", "
            + "}";
    }

```

Assets/Scripts/Config/DrChapter.cs ... Assets/Scripts/Config/DrLevel.cs

```

    }
}
}

//-----
// <auto-generated>
//     This code was generated by a tool.
//     Changes to this file may cause incorrect behavior and will be lost if
//     the code is regenerated.
// </auto-generated>
//-----

using Luban;
using SimpleJSON;

namespace cfg
{
public sealed partial class DrLevel : Luban.BeanBase
{
    public DrLevel(JSONNode _buf)
    {
        { if(!_buf["id"].IsNumber) { throw new SerializationException(); } Id = _buf["id"]; }
        { if(!_buf["next_level"].IsNumber) { throw new SerializationException(); } NextLevel = _buf["next_level"]; }
        { if(!_buf["level_figure"].IsString) { throw new SerializationException(); } LevelFigure = _buf["level_figure"]; }
        { if(!_buf["level_figure_x"].IsNumber) { throw new SerializationException(); } LevelFigureX = _buf["level_figure_x"]; }
        { if(!_buf["level_figure_y"].IsNumber) { throw new SerializationException(); } LevelFigureY = _buf["level_figure_y"]; }
        { if(!_buf["difficulty_tier"].IsNumber) { throw new SerializationException(); } DifficultyTier = _buf["difficulty_tier"]; }
        { if(!_buf["out_of_place_number"].IsNumber) { throw new SerializationException(); } OutOfPlaceNumber = _buf["out_of_place_numbe ...
        { var __json0 = _buf["m_distance_range"]; if(!__json0.IsArray) { throw new SerializationException(); } MDistanceRange = new Syst ...
    }

    public static DrLevel DeserializeDrLevel(JSONNode _buf)
    {
        return new DrLevel(_buf);
    }

    /// <summary>
    /// ID
    /// </summary>
    public readonly int Id;
    /// <summary>
    /// 下一章章节id
    /// </summary>
    public readonly int NextLevel;
    /// <summary>
    /// 关卡图片<br/>（配置资源名称）
    /// </summary>
    public readonly string LevelFigure;

```

Assets/Scripts/Config/DrLevel.cs ... Assets/Scripts/Config/Tables.cs

```

    /// <summary>
    /// x轴切分数量
    /// </summary>
    public readonly int LevelFigureX;
    /// <summary>
    /// y轴切分数量
    /// </summary>
    public readonly int LevelFigureY;
    /// <summary>
    /// 关卡难度<br/>1. 普通<br/>2. 困难
    /// </summary>
    public readonly int DifficultyTier;
    /// <summary>
    /// 至多 {0} 块在原位
    /// </summary>
    public readonly int OutOfPlaceNumber;
    /// <summary>
    /// 总曼哈顿距离范围<br/>（左闭右闭）
    /// </summary>
    public readonly System.Collections.Generic.List<int> MDistanceRange;

    public const int __ID__ = -737222794;
    public override int GetTypeId() => __ID__;

    public void ResolveRef(Tables tables)
    {
    }

    public override string ToString()
    {
        return "{ "
            + "id:" + Id + ", "
            + "nextLevel:" + NextLevel + ", "
            + "levelFigure:" + LevelFigure + ", "
            + "levelFigureX:" + LevelFigureX + ", "
            + "levelFigureY:" + LevelFigureY + ", "
            + "difficultyTier:" + DifficultyTier + ", "
            + "outOfPlaceNumber:" + OutOfPlaceNumber + ", "
            + "mDistanceRange:" + Luban.StringUtil.CollectionToString(MDistanceRange) + ", "
            + "}";
    }
}
}

//-----
// <auto-generated>
//     This code was generated by a tool.
//     Changes to this file may cause incorrect behavior and will be lost if
//     the code is regenerated.

```

Assets/Scripts/picmgr.cs

```

        //          int rnx = int.Parse(rnParts[0]);
        //          int rny = int.Parse(rnParts[1]);
        //          bool rnIsCorner = (rnx == 0 && rny == (height - 1)) || (rnx == (width - 1) && rny == (height - 1)) || (rnx ...
        //          if (!rnIsCorner) bl.SetResourceAndLoad("b");
        //      }
        //  }
        //  }
        //}
    }
}
/*
 * 检查胜利
 */
internal void CheckSucess()
{
    bool suc = true;
    foreach (Transform child in transform)
    {
        var c = child.GetComponent<DraggableGridItem>();
        if (c == null) continue;

        // 解析当前卡牌名称, 格式为 "GridCell_x_y"
        string[] parts = child.name.Replace("GridCell_", "").Split('_');
        if (parts.Length != 2) continue;

        int x, y;
        if (!int.TryParse(parts[0], out x) || !int.TryParse(parts[1], out y)) continue;

        // 计算预期的邻居名称
        string expectedRightName = "GridCell_" + (x + 1) + "_" + y;
        string expectedLeftName = "GridCell_" + (x - 1) + "_" + y;
        string expectedTopName = "GridCell_" + x + "_" + (y + 1);
        string expectedBottomName = "GridCell_" + x + "_" + (y - 1);

        // 查找实际的邻居
        DraggableGridItem rightNeighbor = null;
        DraggableGridItem leftNeighbor = null;
        DraggableGridItem topNeighbor = null;
        DraggableGridItem bottomNeighbor = null;

        foreach (Transform otherChild in transform)
        {
            if (otherChild == child) continue;

            var otherItem = otherChild.GetComponent<DraggableGridItem>();
            if (otherItem == null) continue;

            // 检查相邻位置
            RectTransform otherRect = otherChild.GetComponent<RectTransform>();
            RectTransform currentRect = child.GetComponent<RectTransform>();

```

Assets/Scripts/picmgr.cs

```
        if (otherRect == null || currentRect == null) continue;

        // 计算位置差异
        Vector2 diff = otherRect.anchoredPosition - currentRect.anchoredPosition;
        float threshold = 1.0f; // 位置差异阈值

        if (Mathf.Abs(diff.x - currentRect.sizeDelta.x) < threshold && Mathf.Abs(diff.y) < threshold)
        {
            rightNeighbor = otherItem;
        }
        else if (Mathf.Abs(diff.x + currentRect.sizeDelta.x) < threshold && Mathf.Abs(diff.y) < threshold)
        {
            leftNeighbor = otherItem;
        }
        else if (Mathf.Abs(diff.y - currentRect.sizeDelta.y) < threshold && Mathf.Abs(diff.x) < threshold)
        {
            topNeighbor = otherItem;
        }
        else if (Mathf.Abs(diff.y + currentRect.sizeDelta.y) < threshold && Mathf.Abs(diff.x) < threshold)
        {
            bottomNeighbor = otherItem;
        }
    }

    // 检查邻居名称是否正确
    bool isRightCorrect = (rightNeighbor == null) || (rightNeighbor.gameObject.name == expectedRightName);
    bool isLeftCorrect = (leftNeighbor == null) || (leftNeighbor.gameObject.name == expectedLeftName);
    bool isTopCorrect = (topNeighbor == null) || (topNeighbor.gameObject.name == expectedTopName);
    bool isBottomCorrect = (bottomNeighbor == null) || (bottomNeighbor.gameObject.name == expectedBottomName);

    // 如果任何一个方向不正确, 设置suc为false
    if (!isRightCorrect || !isLeftCorrect || !isTopCorrect || !isBottomCorrect)
    {
        suc = false;
        break;
    }
}

if (suc)
{
    var t = transform.GetComponent<RectTransform>();
    if (t != null)
    {
        // 向上移动100像素
        Vector2 currentPos = t.anchoredPosition;
        t.DOAnchorPos(new Vector2(currentPos.x, currentPos.y + 100f), 0.5f).onComplete=() =>
        {
            // 显示胜利界面
            for(int i = 0;i< transform.childCount; i++)
            {
```

Assets/Scripts/picmgr.cs

```
        var x= transform.GetChild(i);
        x.GetComponent<DraggableGridItem>().enabled=false;
    }

    Main.DispEvent("show_next", curlevel);
};
}
}
}

// 根据名称查找网格项
private DraggableGridItem FindGridItemByName(string name)
{
    foreach (Transform child in transform)
    {
        if (child.name == name)
        {
            return child.GetComponent<DraggableGridItem>();
        }
    }
    return null;
}

// 在指定的anchoredPosition附近查找网格项（根据实际位置检测）
private DraggableGridItem FindGridItemAtAnchoredPosition(Vector2 anchoredPosition, List<DraggableGridItem> gridItems, float threshold)
{
    foreach (DraggableGridItem item in gridItems)
    {
        RectTransform rt = item.GetComponent<RectTransform>();
        if (rt == null) continue;
        if (Vector2.Distance(rt.anchoredPosition, anchoredPosition) <= threshold)
        {
            return item;
        }
    }
    return null;
}

// 根据名称查找子对象
private GameObject FindChildByName(GameObject parent, string name)
{
    Transform child = parent.transform.Find(name);
    return child != null ? child.gameObject : null;
}

// 刷新所有卡牌的位置索引
public void RefreshAllPositionIndices()
{
    foreach (Transform child in transform)
```

Assets/Scripts/picmgr.cs

```
{
    DraggableGridItem item = child.GetComponent<DraggableGridItem>();
    if (item != null)
    {
        // 根据当前位置更新位置索引
        int x = Mathf.RoundToInt(item.rectTransform.anchoredPosition.x / carWid);
        int y = Mathf.RoundToInt(item.rectTransform.anchoredPosition.y / carHei);
        item.PositionIndex = x * this.height + y;
        var originalPosText = item.GetComponentInChildren<TextMeshProUGUI>();
        if(originalPosText != null)
        {
            originalPosText.text = item.PositionIndex.ToString();
        }
    }
}

// 新增方法：计算两点之间的曼哈顿距离（用于网格距离计算）
private int CalculateManhattanDistance(int x1, int y1, int x2, int y2)
{
    return Mathf.Abs(x1 - x2) + Mathf.Abs(y1 - y2);
}

// 修改后的ShuffleGridPositions方法
public void ShuffleGridPositions(int level = 1, int maxKeepCount = -1, bool isHard = false)
{
    // 收集所有子对象
    List<RectTransform> children = new List<RectTransform>();
    List<Vector2> originalPositions = new List<Vector2>();
    List<Vector2Int> originalGridCoords = new List<Vector2Int>(); // 存储原始网格坐标

    foreach (Transform child in transform)
    {
        RectTransform rect = child.GetComponent<RectTransform>();
        if (rect != null)
        {
            children.Add(rect);
            originalPositions.Add(rect.anchoredPosition);

            // 解析原始网格坐标
            string[] parts = child.name.Replace("GridCell_", "").Split('_');
            if (parts.Length == 2)
            {
                int x, y;
                if (int.TryParse(parts[0], out x) && int.TryParse(parts[1], out y))
                {
                    originalGridCoords.Add(new Vector2Int(x, y));
                    continue;
                }
            }
        }
    }
}
```

Assets/Scripts/picmgr.cs

```
        originalGridCoords.Add(new Vector2Int(-1, -1)); // 解析失败的情况
    }
}

// 如果子对象数量不足2个，无法打乱
if (children.Count < 2)
{
    return;
}

// 获取网格单元的尺寸（从第一个子对象获取）
Vector2 cellSize = children[0].sizeDelta;

// 创建可用位置列表（初始为所有位置）
List<int> availableIndices = new List<int>();
for (int i = 0; i < children.Count; i++)
{
    availableIndices.Add(i);
}

// 计算网格的宽高
int gridWidth = width;
int gridHeight = height;

// 计算默认maxKeepCount：总卡牌数的30%
if (maxKeepCount == -1)
{
    maxKeepCount = Mathf.Max(0, Mathf.RoundToInt(children.Count * 0.3f));
}

// 确保maxKeepCount在有效范围内
maxKeepCount = Mathf.Clamp(maxKeepCount, 0, children.Count - 1);

// 随机选择要保留在原位的卡牌数量（0到maxKeepCount之间）
// 使用关卡编号和难度作为随机种子，确保每次结果一致
int seed = level * 1000 + (isHard ? 1 : 0);
System.Random rng = new System.Random(seed);
int cardsToKeep = rng.Next(0, maxKeepCount + 1);

// 随机选择要保留在原位的卡牌索引
List<int> keepIndices = new List<int>();
List<int> allIndices = new List<int>();
for (int i = 0; i < children.Count; i++)
{
    allIndices.Add(i);
}

// 打乱allIndices列表，使用固定种子确保结果一致
System.Random shuffleRng = new System.Random(seed + 1);
for (int i = allIndices.Count - 1; i > 0; i--)
{

```

Assets/Scripts/picmgr.cs

```
        int j = shuffleRng.Next(0, i + 1);
        int temp = allIndices[i];
        allIndices[i] = allIndices[j];
        allIndices[j] = temp;
    }

    // 选择前cardsToKeep个索引作为要保留的卡牌
    for (int i = 0; i < cardsToKeep; i++)
    {
        keepIndices.Add(allIndices[i]);
        // 从可用位置列表中移除这些索引，它们将保持原位
        availableIndices.Remove(allIndices[i]);
    }

    // 为剩余的卡牌分配新位置
    List<int> targetIndices = new List<int>(availableIndices);

    // 困难模式下，尝试让卡牌离原位置更远
    if (isHard && originalGridCoords.Count == children.Count)
    {
        // 为困难模式创建一个偏向于远距离交换的映射
        System.Random distanceRng = new System.Random(seed + 3);

        // 多次尝试优化位置，使卡牌移动得更远
        for (int attempt = 0; attempt < 3; attempt++)
        {
            for (int i = 0; i < targetIndices.Count - 1; i++)
            {
                int currentIndex = availableIndices[i];
                int currentTarget = targetIndices[i];

                // 检查当前位置和目标位置的距离
                if (currentIndex < originalGridCoords.Count && currentTarget < originalGridCoords.Count &&
                    originalGridCoords[currentIndex].x >= 0 && originalGridCoords[currentTarget].x >= 0)
                {
                    int currentDistance = CalculateManhattanDistance(
                        originalGridCoords[currentIndex].x, originalGridCoords[currentIndex].y,
                        originalGridCoords[currentTarget].x, originalGridCoords[currentTarget].y);

                    // 寻找一个更远的目标位置
                    for (int j = i + 1; j < targetIndices.Count; j++)
                    {
                        int otherIndex = availableIndices[j];
                        int otherTarget = targetIndices[j];

                        if (otherIndex < originalGridCoords.Count && otherTarget < originalGridCoords.Count &&
                            originalGridCoords[otherIndex].x >= 0 && originalGridCoords[otherTarget].x >= 0)
                        {
                            int newDistance1 = CalculateManhattanDistance(
                                originalGridCoords[currentIndex].x, originalGridCoords[currentIndex].y,
```

Assets/Scripts/picmgr.cs

```

        originalGridCoords[otherTarget].x, originalGridCoords[otherTarget].y);

        int newDistance2 = CalculateManhattanDistance(
            originalGridCoords[otherIndex].x, originalGridCoords[otherIndex].y,
            originalGridCoords[currentTarget].x, originalGridCoords[currentTarget].y);

        // 如果交换后两个卡牌都离原位置更远，则进行交换
        if (newDistance1 > currentDistance && newDistance2 >
            CalculateManhattanDistance(
                originalGridCoords[otherIndex].x, originalGridCoords[otherIndex].y,
                originalGridCoords[otherTarget].x, originalGridCoords[otherTarget].y))
        {
            // 交换目标索引
            int temp = targetIndices[i];
            targetIndices[i] = targetIndices[j];
            targetIndices[j] = temp;
            break;
        }
    }
}

}

}

}

}

else
{
    // 普通模式或无法计算网格坐标时，使用原有随机方式
    System.Random targetRng = new System.Random(seed + 2);
    for (int i = targetIndices.Count - 1; i > 0; i--)
    {
        int j = targetRng.Next(0, i + 1);
        int temp = targetIndices[i];
        targetIndices[i] = targetIndices[j];
        targetIndices[j] = temp;
    }
}

// 应用位置变化
for (int i = 0; i < children.Count; i++)
{
    if (keepIndices.Contains(i))
    {
        // 保持原位
        continue;
    }

    // 找到当前索引在availableIndices中的位置
    int indexInAvailable = availableIndices.IndexOf(i);
    if (indexInAvailable >= 0 && indexInAvailable < targetIndices.Count)
    {

```

Assets/Scripts/picmgr.cs ... Assets/Scripts/PlayerData.cs

```
        int targetIndex = targetIndices[indexInAvailable];
        children[i].anchoredPosition = originalPositions[targetIndex];
    }
}

RefreshAllPositionIndices();

// 更新边框显示状态
UpdateBorderVisibility();
}
}

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using System.IO;
using LitJson;
using System;

public class opend
{
    JsonData _data;
    public opend(JsonData data)
    {
        _data = data;
    }

    internal bool Opened(int id)
    {
        bool ret = false;

        for(int i = 0; i < _data.Count; i++)
        {
            if(int.Parse(_data[i].ToString()) == id)
            {
                ret = true;
                break;
            }
        }
        return ret;
    }

    internal void Open(int id)
    {
        if(!Opened(id))
        {
            _data.Add(id.ToString());
        }
    }
}

public class GameData
{

```

Assets/Scripts/PlayerData.cs

```
internal int levelid
{
    get
    {
        if(data.Has("levelid"))
        {
            return int.Parse( data["levelid"].ToString());
        }
        return 100001;
    }
    set
    {
        data["levelid"] = value.ToString();
        Main.DispEvent("onLevelChange");
    }
}

// power相关字段和方法
internal int power
{
    get
    {
        int currentpower = int.Parse(data.Has("power") ? data["power"].ToString() : "100");
        int maxpower = 100;

        // 计算自动恢复的power
        long lastUpdateTime = lastpowerUpdateTime;
        long now = DateTime.Now.Ticks / TimeSpan.TicksPerSecond;
        long elapsedSeconds = now - lastUpdateTime;

        int recoveredpower = (int)(elapsedSeconds / (15 * 60)); // 每15分钟恢复1点power
        if (recoveredpower > 0)
        {
            currentpower = Math.Min(maxpower, currentpower + recoveredpower);
            data["power"] = Math.Min(100, currentpower).ToString();
            lastpowerUpdateTime = now;
        }

        return currentpower;
    }
    set
    {
        data["power"] = Math.Min(100, value).ToString();
        lastpowerUpdateTime = DateTime.Now.Ticks / TimeSpan.TicksPerSecond;
        Main.DispEvent("onpowerChange");
    }
}

internal int coin
{

```

Assets/Scripts/PlayerData.cs

```
    get
    {
        return int.Parse(data.Has("coin") ? data["coin"].ToString() : "1280");
    }
    set
    {
        data["coin"] = Math.Max(0, value).ToString();
    }
}
```

```
internal long lastpowerUpdateTime
{
    get
    {
        if(data.Has("lastpowerUpdateTime"))
        {
            return long.Parse(data["lastpowerUpdateTime"].ToString());
        }
        return DateTime.Now.Ticks / TimeSpan.TicksPerSecond;
    }
    set
    {
        data["lastpowerUpdateTime"] = value.ToString();
    }
}
```

```
// 检查是否有足够的power
public bool hasEnoughpower(int cost = 10)
{
    return power >= cost;
}
```

```
// 消耗power
public bool 消耗power(int cost = 10)
{
    if(hasEnoughpower(cost))
    {
        power -= cost;
        return true;
    }
    return false;
}
```

```
opend op;
```

```
JsonData data;
internal int currChapter
{
    get
    {

```

Assets/Scripts/PlayerData.cs

```
        if (data.Has("currChapter"))
        {
            return int.Parse(data["currChapter"].ToString());
        }
        return 1;
    }
    set
    {
        data["currChapter"] = value.ToString();
        Main.DispEvent("onChapterChange");
    }
}
```

```
public GameData(JsonData _data)
{
    data = _data;
    op = new opend(openeddata);
}
JsonData openeddata
{
    get
    {
        if (!data.Has("opened"))
        {
            var x = new JsonData();
            x.SetJsonType(JsonType.Array);
            data["opened"] = x;
        }
        if (!data["opened"].IsArray)
        {
            var x = new JsonData();
            x.SetJsonType(JsonType.Array);
            data["opened"] = x;
        }
        return data["opened"];
    }
}
```

```
public bool isOpened(int id)
{
    return op.Opened(id);
}
public void Open(int id)
{
    op.Open(id);
    Main.DispEvent("onLevelChange");
}
public JsonData getData()
{
    return data;
}
```

Assets/Scripts/PlayerData.cs

```
    }

    internal void FillPower()
    {
        power += 10;
    }
}

public class PlayerData : MonoBehaviour
{
    bool chapterOpend()
    {
        bool ret = true;
        int cid = gd.currChapter;
        var sst = datamgr.Instance.GetChapter(cid);
        for (int i = 0; i < sst.LevelId.Count; i++)
        {
            var xt = sst.LevelId[i];
            if (!gd.isOpened(xt))
            {
                ret = false;
                break;
            }
        }
        return ret;
    }

    // Start is called before the first frame update
    public static GameData gd;
    private void Awake()
    {
        loadData();
        Main.RegistEvent("onLevelChange", (x) =>
        {
            if (chapterOpend())
            {
                gd.currChapter++;
            }

            saveData();
            return 1;
        });
        Main.RegistEvent("onpowerChange", (x) =>
        {
            saveData();
            return null;
        });
    }

    void loadData()
    {
        var pa = Application.persistentDataPath + "/playerData.json";
```

Assets/Scripts/PlayerData.cs

```
        if(File.Exists(pa))
        {
            var json = File.ReadAllText(pa);
            JsonData data = null;
            try
            {
                data = JsonSerializer.ToObject<JsonData>(json);
            }
            catch (Exception e)
            {
                Debug.LogError(e.Message);
                data = new JsonData();
            }
            if(data==null){
                data = new JsonData();
            }
            gd = new GameData(data);
        }
        else
        {
            gd = new GameData(new JsonData());
        }
    }

    public void saveData()
    {
        var pa = Application.persistentDataPath+"/playerData.json";
        var json = JsonSerializer.ToJson(gd.getData());
        File.WriteAllText(pa, json);
    }
    // Update is called once per frame
    void Update()
    {
    }
}

public static class ExtendClass
{
    public static bool Has(this JsonData data, string vx)
    {
        if (data == null || vx == null || !data.IsObject)
        {
            return false;
        }

        try
        {
            IDictionary dict = data as IDictionary;
            if (dict == null)
```

Assets/Scripts/PlayerData.cs ... Assets/Scripts/recttools.cs

```
        {
            return false;
        }

        foreach (object key in dict.Keys)
        {
            if (key != null && key.ToString() == vx)
            {
                return true;
            }
        }
    }
    catch
    {
        return false;
    }

    return false;
}
}

using UnityEngine;

public static class RectTransformExtension
{
    /// <summary>
    /// 判断子RectTransform是否超出父RectTransform的范围
    /// </summary>
    /// <param name="child">子节点RectTransform</param>
    /// <param name="parent">父节点RectTransform（必须是child的直接或间接父节点）</param>
    /// <param name="checkPartial">true=判断是否部分超出；false=判断是否完全超出</param>
    /// <returns>是否超出范围</returns>
    public static bool IsOutOfParentBounds(this RectTransform child, RectTransform parent, bool checkPartial = true)
    {
        // 1. 校验父节点关系（避免无效计算）
        if (!child.IsChildOf(parent))
        {
            Debug.LogWarning("子节点不是父节点的层级子节点，无法判断边界！");
            return true;
        }

        // 2. 获取父节点的本地矩形（pivot不影响，直接用rect属性）
        Rect parentRect = parent.rect;

        // 3. 获取子节点在父节点本地坐标系下的矩形
        Rect childLocalRect = GetRectInParentSpace(child, parent);

        // 4. 判断是否超出：根据checkPartial选择逻辑（修正Overlaps参数问题）
        if (checkPartial)
        {
            // 部分超出：子节点矩形与父节点矩形不完全包含（即有部分在父节点外）
        }
    }
}
```

Assets/Scripts/recttools.cs

```

        // 逻辑：父节点矩形无法完全包含子节点矩形 → 超出
        return !parentRect.Contains(childLocalRect.min) ||
            !parentRect.Contains(childLocalRect.max) ||
            !parentRect.Contains(new Vector2(childLocalRect.xMin, childLocalRect.yMax)) ||
            !parentRect.Contains(new Vector2(childLocalRect.xMax, childLocalRect.yMin));
    }
    else
    {
        // 完全超出：子节点矩形与父节点矩形无任何重叠
        return !parentRect.Overlaps(childLocalRect);
    }
}

/// <summary>
/// 判断世界坐标点是否在 RectTransform 范围内（支持旋转、缩放、任意锚点）
/// </summary>
/// <param name="rectTransform">目标 RectTransform (UI节点)</param>
/// <param name="worldPoint">世界坐标系下的点（如3D物体位置、UI世界坐标）</param>
/// <returns>点是否在范围内</returns>
public static bool IsWorldPointInBounds(this RectTransform rectTransform, Vector3 worldPoint)
{
    // 将世界坐标转换为 RectTransform 的本地坐标（忽略Z轴，UI是2D平面）
    Vector2 localPoint = rectTransform.InverseTransformPoint(worldPoint);
    // 判断本地坐标是否在 RectTransform 的本地矩形内
    return rectTransform.rect.Contains(localPoint);
}

/// <summary>
/// 判断屏幕坐标点是否在 RectTransform 范围内（适用于点击、触摸检测）
/// </summary>
/// <param name="rectTransform">目标 RectTransform (UI节点)</param>
/// <param name="screenPoint">屏幕坐标系下的点（如 Input.mousePosition）</param>
/// <param name="camera">Canvas 对应的相机（Screen Space - Overlay 模式传 null）</param>
/// <returns>点是否在范围内</returns>
public static bool IsScreenPointInBounds(this RectTransform rectTransform, Vector2 screenPoint, Camera camera = null)
{
    // 将屏幕坐标转换为 RectTransform 的本地坐标
    if (RectTransformUtility.ScreenPointToLocalPointInRectangle(
        rectTransform, screenPoint, camera, out Vector2 localPoint))
    {
        // 转换成功，判断是否在本地矩形内
        return rectTransform.rect.Contains(localPoint);
    }
    // 转换失败（点不在相机可视范围内），返回false
    return false;
}

/// <summary>
/// 获取子节点在父节点本地坐标系下的Rect
/// </summary>
private static Rect GetRectInParentSpace(RectTransform child, RectTransform parent)
{

```

Assets/Scripts/recttools.cs

```
// 存储子节点的4个世界坐标角点（顺序：左下、右下、右上、左上）
Vector3[] childWorldCorners = new Vector3[4];
child.GetWorldCorners(childWorldCorners);

// 存储转换后的父节点本地坐标角点
Vector2[] childLocalCorners = new Vector2[4];
for (int i = 0; i < 4; i++)
{
    // 将世界坐标转换为父节点的本地坐标（忽略Z轴，UI是2D平面）
    childLocalCorners[i] = parent.InverseTransformPoint(childWorldCorners[i]);
}

// 计算子节点本地矩形的边界（min/max坐标）
float xMin = Mathf.Min(childLocalCorners[0].x, childLocalCorners[1].x, childLocalCorners[2].x, childLocalCorners[3].x);
float xMax = Mathf.Max(childLocalCorners[0].x, childLocalCorners[1].x, childLocalCorners[2].x, childLocalCorners[3].x);
float yMin = Mathf.Min(childLocalCorners[0].y, childLocalCorners[1].y, childLocalCorners[2].y, childLocalCorners[3].y);
float yMax = Mathf.Max(childLocalCorners[0].y, childLocalCorners[1].y, childLocalCorners[2].y, childLocalCorners[3].y);

// 返回子节点在父节点本地的Rect
return new Rect(xMin, yMin, xMax - xMin, yMax - yMin);
}

/// <summary>
/// 可选：将子节点限制在父节点范围内（超出时自动回弹）
/// </summary>
public static void ConstrainToParent(this RectTransform child, RectTransform parent)
{
    Rect parentRect = parent.rect;
    Rect childLocalRect = GetRectInParentSpace(child, parent);

    // 计算需要偏移的距离（确保子节点完全在父节点内）
    float offsetX = 0;
    float offsetY = 0;

    // 左边界超出
    if (childLocalRect.xMin < parentRect.xMin)
        offsetX = parentRect.xMin - childLocalRect.xMin;
    // 右边界超出
    else if (childLocalRect.xMax > parentRect.xMax)
        offsetX = parentRect.xMax - childLocalRect.xMax;

    // 下边界超出
    if (childLocalRect.yMin < parentRect.yMin)
        offsetY = parentRect.yMin - childLocalRect.yMin;
    // 上边界超出
    else if (childLocalRect.yMax > parentRect.yMax)
        offsetY = parentRect.yMax - childLocalRect.yMax;

    // 应用偏移（在父节点本地坐标系下调整子节点位置）
    if (offsetX != 0 || offsetY != 0)
```

Assets/Scripts/recttools.cs ... Assets/Scripts/ui/frm_chapter.cs

```

        {
            Vector2 localPos = child.localPosition;
            child.localPosition = new Vector2(localPos.x + offsetX, localPos.y + offsetY);
        }
    }
}

using System;
using System.Collections;
using System.Collections.Generic;
using TMPro;
using UnityEngine;
using UnityEngine.UI;
using TMPro;

public class frm_chapter : frmbase
{
    public RectTransform chaptercontent;
    public Button btn;
    public Button btnsetup;
    public Button btn_addpower;
    public TextMeshProUGUI staminaText, levelname; // power显示文本组件
    public power mpower;
    private void Awake()
    {
        Main.RegistEvent("gamebegin", (x) =>
        {
            brushChapterContent();
            show();
            UpdateStaminaDisplay();
            return 1;
        });

        // 注册power变化事件
        Main.RegistEvent("onpowerChange", (x) =>
        {
            UpdateStaminaDisplay();
            return null;
        });

        Main.RegistEvent("level_next", (x) =>
        {
            PlayerData.gd.levelid = datamgr.Instance.GetLevel(PlayerData.gd.levelid).NextLevel;
            brushChapterContent();
            show();
            return 1;
        });

        Main.RegistEvent("onChapterChange", (x) =>
        {
            brushChapterContent();
            show();
            return 1;
        });
    }
}

```

Assets/Scripts/ui/frm_chapter.cs

```
    });
    Main.RegistEvent("level_back", (x) =>
    {
        show();
        return null;
    });
    btn.onClick.AddListener(() =>
    {
        if (!isTurning())
        {
            int ret =(int)Main.DispEvent("level_play", PlayerData.gd.levelid);
            if (ret==1)
            {
                hide();
            }
        }
    });
    btn_addpower.onClick.AddListener(() =>
    {
        PlayerData.gd.FillPower();
    });
    btnsetup.onClick.AddListener(() =>
    {
        Main.DispEvent("show_setup",null);
    });
}

private bool isTurning()
{
    bool ret = false;
    for(int i=0;i< chaptercontent.childCount; i++)
    {
        var c = chaptercontent.GetChild(i);
        if (c.GetComponent<card>().isTurning)
        {
            ret = true;
            break;
        }
    }
    return ret;
}

private void Update()
{
    // 检测屏幕尺寸变化，自动调整chaptercontent大小
    // ResizeChapterContent();
}

private void ResizeChapterContent()
{

```

Assets/Scripts/ui/frm_chapter.cs

```
// 获取屏幕尺寸
float screenWidth = Screen.width;
float screenHeight = Screen.height;

//// 设置边界边距（最低50像素）
float margin = 130f;

//// 计算可用空间
float availableWidth = screenWidth - 2 * margin;
float availableHeight = screenHeight - 2 * margin;

//// 9:16宽高比的目标宽高
float targetWidth, targetHeight;

//// 根据可用空间计算最佳尺寸
if (availableWidth / availableHeight > 9f / 16f)
{
    // 以高度为基准
    targetHeight = availableHeight;
    targetWidth = targetHeight * 9f / 16f;
}
else
{
    // 以宽度为基准
    targetWidth = availableWidth;
    targetHeight = targetWidth * 16f / 9f;
}

float wside = (screenWidth - targetWidth) / 2;
float hside = (screenHeight - targetHeight) / 2;
chaptercontent.offsetMin = new Vector2(wside, hside); // Left和Bottom (offsetMin = (Left, Bottom))
chaptercontent.offsetMax = new Vector2(-wside, -hside); // Right和Top (offsetMax = (-Right, -Top))

//// 设置chaptercontent的尺寸
//chaptercontent.sizeDelta = new Vector2(targetWidth, targetHeight);

//// 设置chaptercontent居中
//chaptercontent.anchoredPosition = Vector2.zero;
}

protected override void OnShow()
{
    base.OnShow();
    //ResizeChapterContent();
    UpdateStaminaDisplay();
}

private void brushChapterContent()
{
    var chapter = datamgr.Instance.GetChapter(PlayerData.gd.currChapter);
```

Assets/Scripts/ui/frm_chapter.cs

```
var pic = Resources.Load(chapter.ChapterFigure) as Texture;

levelname.text= $"Level { PlayerData.gd.levelid%10000}";
// 调整chaptercontent大小
//ResizeChapterContent();

// 清除现有的子对象（使用倒序删除避免遍历时修改集合的问题）
for (int i = chaptercontent.childCount - 1; i >= 0; i--)
{
    Transform child = chaptercontent.GetChild(i);
    // 在编辑器中使用DestroyImmediate, 在运行时使用Destroy
    #if UNITY_EDITOR
    if (Application.isPlaying)
    {
        Destroy(child.gameObject);
    }
    else
    {
        DestroyImmediate(child.gameObject);
    }
    #else
    Destroy(child.gameObject);
    #endif
}

// 获取父容器的尺寸
Vector2 rectSize = chaptercontent.rect.size;

// 计算每个格子的尺寸
float cellWidth = (rectSize.x / chapter.ChapterFigureX);
float cellHeight = (rectSize.y / chapter.ChapterFigureY);

// 外层循环为行（从上到下），内层循环为列（从左到右）
for(int j=0; j< chapter.ChapterFigureY; j++)
{
    for(int i=0; i< chapter.ChapterFigureX; i++)
    {
        // 索引计算修改为：行号 * 每行元素数 + 列号，确保从左到右、从上到下的顺序
        int idx = j * chapter.ChapterFigureX + i;
        int levelid = chapter.LevelId[idx];

        // 创建一个空的游戏对象作为格子
        GameObject cellObject = GameObject.Instantiate(Resources.Load("levelpic")) as GameObject;// new GameObject("ChapterCell_ ...
        cellObject.transform.SetParent(chaptercontent, false);
        card pcard = cellObject.GetComponent<card>();
        pcard.levelid = levelid;
        // 添加RectTransform组件
        RectTransform cellRect = cellObject.GetComponent<RectTransform>();
```

Assets/Scripts/ui/frm_chapter.cs ... Assets/Scripts/ui/frm_game.cs

```

        // 设置位置和尺寸
        cellRect.anchorMin = new Vector2(0, 0);
        cellRect.anchorMax = new Vector2(0, 0);
        cellRect.pivot = new Vector2(0.5f, 0);
        cellRect.anchoredPosition = new Vector2((i+0.5f) * cellWidth, (chapter.ChapterFigureY- 1 - j) * cellHeight);
        cellRect.sizeDelta = new Vector2(cellWidth-2, cellHeight-2);
        // 添加RawImage组件
        RawImage rawImage = cellObject.GetComponent<RawImage>();

        pcard.uvX = (float)i / chapter.ChapterFigureX;
        pcard.uvY = (float)(j) / chapter.ChapterFigureY;
        pcard.uvWidth = 1.0f / chapter.ChapterFigureX;
        pcard.uvHeight = 1.0f / chapter.ChapterFigureY;
        pcard.texture = pic;
        pcard.Load();
        pcard.level.text = levelid.ToString();
    }
}

// 更新power显示
private void UpdateStaminaDisplay()
{
    //string powerText;
    //if (PlayerData.gd.power < 1000)
    //{
    //    powerText = PlayerData.gd.power.ToString();
    //}
    //else
    //{
    //    float kValue = PlayerData.gd.power / 1000f;
    //    powerText = $" {kValue:F1}K";
    //}

    float v = (float)PlayerData.gd.power / 100f;
    mpower.Value = v;
}
}

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using TMPro;
using System;

public class frm_game : frmbase

```

Assets/Scripts/ui/frm_game.cs

```

{
    public picmgr mgr;
    public TextMeshProUGUI level;
    public Button next;
    public Button back;
    public Button setup;
    private void Awake()
    {
        Main.RegistEvent("level_play", (x) =>
        {
            // 检查并消耗power
            if (!PlayerData.gd.hasEnoughpower(10))
            {
                Debug.Log("power不足，无法开始游戏");
                // 可以在这里添加power不足的提示
                Main.DispEvent("event_msg", "power不足，无法开始游戏");
                return 0;
            }

            // 消耗10点power
            PlayerData.gd.消耗power(10);

            next.gameObject.SetActive(false);
            var rectTrans = mgr.GetComponent<RectTransform>();
            // 2. 设置边距(Left/Right/Top/Bottom)和图片大小一致
            rectTrans.offsetMin = new Vector2(82.82125f, 124.4235f); // Left和Bottom, offsetMin = (Left, Bottom)
            rectTrans.offsetMax = new Vector2(-82.82125f, -208.2435f); // Right和Top, offsetMax = (-Right, -Top)
            mgr.ResizeChapterContent();
            var leevel = datamgr.Instance.GetLevel((int)x);
            level.text = $"Level { leevel.Id}";
            if (lelevel.DifficultyTier == 2)
            {
                var df = gb.Find("diff");
                df.gameObject.SetActive(true);
                DG.Tweening.DOVirtual.Float(0, 1, .5f, (xvx) =>
                {
                    df.GetComponent<CanvasGroup>().alpha = xvx;

                }).onComplete = () => {

                    DG.Tweening.DOVirtual.DelayedCall(1, () =>
                    {
                        DG.Tweening.DOVirtual.Float(1, 0, .5f, (xvx) =>
                        {
                            df.GetComponent<CanvasGroup>().alpha = xvx;
                        }).onComplete = () =>
                        {
                            df.gameObject.SetActive(false);
                        };
                    });
                });
            }
        });
    }
}

```

Assets/Scripts/ui/frm_game.cs ... Assets/Scripts/ui/frm_login.cs

```

        };
    }
    show();
    StartCoroutine(load(leevel));
    return 1;
});
Main.RegistEvent("level_next", (x) =>
{
    hide();
    return null;
});
next.onClick.AddListener(() =>
{
    Main.SendEvent("level_next");
    hide();
});
back.onClick.AddListener(() =>
{
    Main.SendEvent("level_back");
    hide();
});
setup.onClick.AddListener(() =>
{
    Main.DispEvent("show_setup");
});
}
IEnumerator load(cfg.DrLevel leevel)
{
    yield return StartCoroutine(mgr.LoadLevel(leevel));
}

// Start is called before the first frame update
void Start()
{
}

// Update is called once per frame
void Update()
{
}
}
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using TMPro;
using System;

```

Assets/Scripts/ui/frm_login.cs ... Assets/Scripts/ui/frm_setup.cs

```
public class frm_login : frmbase
{
    public picmgr mgr;
    public TextMeshProUGUI level;
    public Button next;
    public Button back;
    public Button setup;
    private Coroutine openGameCoroutine;

    private void Awake()
    {
        Main.RegistEvent("login_begin", (x) =>
        {
            show();
            if (openGameCoroutine != null)
            {
                StopCoroutine(openGameCoroutine);
            }
            openGameCoroutine = StartCoroutine(OpenChapterAfterDelay());
            return 1;
        });
    }

    // Start is called before the first frame update
    IEnumerator OpenChapterAfterDelay()
    {
        yield return new WaitForSeconds(3f);
        object result = Main.DispEvent("gamebegin");
        if (result is int ret && ret == 1)
        {
            hide();
        }
        openGameCoroutine = null;
    }

    // Update is called once per frame
    void Update()
    {
    }
}

using System;
using System.Collections;
using System.Collections.Generic;
using TMPro;
using UnityEngine;
using UnityEngine.UI;
using System.IO;
```

Assets/Scripts/ui/frm_setup.cs

```
public class frm_setup : frmbase
{
    public Button btnInitialize;
    public Button btnClose;
    public Button btn_buytili;
    private void Awake()
    {
        // 注册按钮点击事件
        btnInitialize.onClick.AddListener(OnInitializeClick);
        btnClose.onClick.AddListener(OnCloseClick);
        //btn_buytili.onClick.AddListener(OnBuyTiliClick);
        // 监听显示事件
        Main.RegistEvent("show_setup", (x) =>
        {
            show();
            return 1;
        });
    }

    private void OnBuyTiliClick()
    {
        PlayerData.gd.power += 100;
    }

    /// <summary>
    /// 关闭设置界面
    /// </summary>
    private void OnCloseClick()
    {
        hide();
    }

    /// <summary>
    /// 初始化游戏数据
    /// </summary>
    private void OnInitializeClick()
    {
        try
        {
            // 删除playerdata.json文件
            string filePath = Application.persistentDataPath + "/playerData.json";
            if (File.Exists(filePath))
            {
                File.Delete(filePath);
                Debug.Log("Player data file deleted successfully.");
            }

            // 重新加载数据
            PlayerData playerData = FindObjectOfType<PlayerData>();
```

Assets/Scripts/ui/frm_setup.cs ... Assets/Scripts/ui/frmbase.cs

```
        if (playerData != null)
        {
            // 调用playerData的loadData方法重新加载数据
            playerData.SendMessage("loadData");
            Debug.Log("Player data reloaded successfully.");
        }

        // 触发事件通知其他组件数据已更新
        Main.DispEvent("onLevelChange");

        Debug.Log("Game initialized successfully.");
        Application.Quit();
    }
    catch (Exception e)
    {
        Debug.LogError("Failed to initialize game data: " + e.Message);
    }
}

using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class frmbase : MonoBehaviour
{
    // Start is called before the first frame update
    private void Awake()
    {

    }

    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {

    }

    public bool isOpen()
    {
        return gb.gameObject.activeSelf;
    }

    protected Transform gb
    {
        get
        {
            return transform.Find("Root");
        }
    }
}
```


Assets/Scripts/ui/frmbase.cs ... Assets/TestParticlePosition.cs

```

    }
}
public void show()
{
    gb.gameObject.SetActive(true);
    if (Application.isPlaying)
    {
        OnShow();
    }
}
protected virtual void OnShow()
{
}
protected virtual void OnHide()
{
}
public virtual void hide()
{
    OnHide();
    gb.gameObject.SetActive(false);
}

public void loadString()
{
    var sx = GetComponentsInChildren<TMPPro.TextMeshProUGUI>(true);
    for (int i = 0; i < sx.Length; i++)
    {
        var x = sx[i];
        //var tx = x.GetComponent<stringinfo>();
        //if (tx == null)
        //{
        //    tx = x.gameObject.AddComponent<stringinfo>();
        //}
        //tx.getString();
    }
}
}

using Coffee.UIExtensions;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

/// <summary>
/// 测试UIParticle中粒子位置获取的脚本
/// </summary>
public class TestParticlePosition : MonoBehaviour
{
    public ParticleSystem uiParticle;

```

Assets/TestParticlePosition.cs ... Assets/UIParticleToUI.cs

```

    public Canvas targetCanvas;
    public Transform debugMarker; // 用于可视化粒子位置的标记

    private ParticleSystem.Particle[] particles;

    void Start()
    {
        if (uiParticle != null)
        {
            particles = new ParticleSystem.Particle[uiParticle.main.maxParticles];
        }
    }

    void Update()
    {
        if (uiParticle == null || targetCanvas == null) return;

        int count = uiParticle.GetParticles(particles);
        if (count > 0)
        {
            // 获取第一个粒子的位置
            Vector3 particleWorldPos = particles[0].position;

            // 显示原始世界坐标
            Debug.Log($"粒子世界坐标: {particleWorldPos}");

            // 转换为屏幕坐标
            Vector2 screenPos = RectTransformUtility.WorldToScreenPoint(null, particleWorldPos);
            Debug.Log($"粒子屏幕坐标: {screenPos}");

            // 如果有标记对象，将其放置在粒子位置
            if (debugMarker != null)
            {
                debugMarker.position = particleWorldPos;
            }
        }
    }
}

using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// UIParticle 粒子坐标转 UI 坐标工具
/// </summary>
public class UIParticleToUI : MonoBehaviour
{
    [Header("必填组件")]
    public ParticleSystem uiParticle; // 你的UIParticle粒子系统
    public Canvas targetCanvas;       // 目标Canvas (UI所在的Canvas)
    public Camera uiCamera;           // 仅Canvas=ScreenSpace-Camera时需要赋值 (Canvas的Render Camera)
}

```

Assets/UIParticleToUI.cs

```

// 测试：获取第0个粒子的UI坐标并打印
void Update()
{
    if (Input.GetKeyDown(KeyCode.Space))
    {
        Vector2 uiCoord = GetParticleUICoord(0);
        Debug.Log($"第0个粒子的UI坐标（Canvas局部）：{uiCoord}");
    }
}

/// <summary>
/// 获取指定索引粒子的UI坐标（Canvas局部，像素单位）
/// </summary>
/// <param name="particleIndex">粒子索引（从0开始）</param>
/// <returns>UI坐标（可直接赋值给RectTransform.anchoredPosition，需注意单位转换）</returns>
public Vector2 GetParticleUICoord(int particleIndex)
{
    // 1. 校验组件
    if (uiParticle == null || targetCanvas == null) return Vector2.zero;

    // 2. 获取粒子数组（复用数组，减少GC）
    ParticleSystem.Particle[] particles = new ParticleSystem.Particle[uiParticle.main.maxParticles];
    int activeCount = uiParticle.GetParticles(particles);
    if (particleIndex < 0 || particleIndex >= activeCount) return Vector2.zero;

    // 3. 粒子坐标 → 世界坐标（适配粒子的Simulation Space）
    Vector3 particleWorldPos = particles[particleIndex].position;

    // 4. 世界坐标 → UI坐标（Canvas局部，适配Canvas渲染模式）
    return WorldPosToUICoord(particleWorldPos);
}

#region 内部辅助方法
/// <summary>
/// 粒子坐标转世界坐标（处理粒子不同的空间模式）
/// </summary>

/// <summary>
/// 世界坐标转UI坐标（Canvas局部，像素单位）
/// </summary>
private Vector2 WorldPosToUICoord(Vector3 worldPos)
{
    Vector2 uiCoord = Vector2.zero;
    RectTransform canvasRect = targetCanvas.transform as RectTransform;

    switch (targetCanvas.renderMode)
    {
        // 最常用：ScreenSpace-Overlay（纯2D UI）
        case RenderMode.ScreenSpaceOverlay:
    
```

Assets/UIParticleToUI.cs

```

        // 世界坐标 → 屏幕坐标（修正Y轴：Screen原点在左下，Canvas在左上）
        Vector2 screenPos = RectTransformUtility.WorldToScreenPoint(null, worldPos);
        screenPos.y = Screen.height - screenPos.y;
        // 屏幕坐标 → Canvas局部坐标
        RectTransformUtility.ScreenPointToLocalPointInRectangle(
            canvasRect, screenPos, null, out uiCoord
        );
        break;

    // ScreenSpace-Camera（带UI相机的模式）
    case RenderMode.ScreenSpaceCamera:
        if (uiCamera == null) uiCamera = targetCanvas.worldCamera;
        // 世界坐标 → 屏幕坐标（依赖UI相机）
        Vector2 screenPosCam = RectTransformUtility.WorldToScreenPoint(uiCamera, worldPos);
        // 屏幕坐标 → Canvas局部坐标
        RectTransformUtility.ScreenPointToLocalPointInRectangle(
            canvasRect, screenPosCam, uiCamera, out uiCoord
        );
        break;

    // WorldSpace（Canvas是3D物体）
    case RenderMode.WorldSpace:
        // 直接将世界坐标转Canvas局部坐标
        Vector3 canvasLocalPos = targetCanvas.transform.InverseTransformPoint(worldPos);
        uiCoord = new Vector2(canvasLocalPos.x, canvasLocalPos.y);
        break;
    }

    return uiCoord;
}

#endregion
public Image followImage;
public Transform followtag;
void LateUpdate()
{
    ParticleSystem.Particle[] particles = new ParticleSystem.Particle[uiParticle.main.maxParticles];
    int activeCount = uiParticle.GetParticles(particles);
    if (activeCount <= 0) return;
    //Vector2 pixelCoord = GetParticleUICoord(0);
    //Vector2 uiUnitCoord = PixelToUIUnit(pixelCoord);
    //followImage.rectTransform.anchoredPosition = uiUnitCoord;
    followtag.position = particles[0].position;
}

// 扩展：UI坐标单位转换（像素 → UI单位，适配anchoredPosition）
public Vector2 PixelToUIUnit(Vector2 pixelCoord)
{
    float refPPU = targetCanvas.GetComponent<CanvasScaler>().referencePixelsPerUnit;
    return pixelCoord / refPPU; // 默认100像素 = 1UI单位
}
}

```